

Fase 2 An lisis Sint ctico

Proyecto de Compiladores I 8 ISC

Docente: Dra. Blanca Guadalupe Estrada Renter a Fecha de entrega: 19-20 de Junio de 2025

Integrantes del Equipo	Matr cula
Jes s Abraham Robledo L pez	284745
Edgar Alejandro Cede o Su rez	262728

1. Descripción General del Proyecto

El presente proyecto implementa la Fase 2 del compilador: un analizador sintáctico descendente recursivo que procesa los tokens generados por el analizador léxico (Fase 1) y construye un árbol Sintáctico Abstracto (AST). El sistema está integrado en un IDE desarrollado en Python con PyQt6, con interfaz visual profesional, múltiples vistas del árbol y terminales de error enriquecidas.

Componentes principales

compiler/lexer.py	Analizador léxico (Fase 1): tokenización con línea y columna
compiler/parser.py	Analizador sintáctico descendente recursivo
compiler/ast_nodes.py	Definición de nodos del AST
ui/main_window.py	IDE con interfaz gráfica PyQt6
ui/syntax_tree.py	Vista gráfica y colapsable del AST
ui/console.py	Terminales de salida con HTML enriquecido

2. Gramática Implementada

La siguiente gramática libre de contexto fue implementada mediante análisis descendente recursivo sin backtracking. Cada producción corresponde directamente a un método del parser.

```
programa -> main { lista_declaracion }
lista_declaracion -> lista_declaracion declaracion | declaracion
declaracion -> declaracion_variable | lista_sentencias
declaracion_var -> tipo identificador ;
identificador -> id | identificador , id
tipo -> int | float | bool
lista_sentencias -> lista_sentencias sentencia | (vacio)
sentencia -> seleccion | iteracion | repeticion | sent_in | sent_out | asignacion
asignacion -> id = sent_expresion
sent_expresion -> expresion ; | ;
seleccion -> if expresion then lista_sentencias [ else lista_sentencias ] end
iteracion -> while expresion lista_sentencias end
repeticion -> do lista_sentencias while expresion
sent_in -> cin >> id ;
sent_out -> cout << salida
salida -> cadena | expresion | cadena << expresion | expresion << cadena
expresion -> expresion_simple [ rel_op expresion_simple ]
rel_op -> < | <= | > | >= | == | !=
expresion_simple -> expresion_simple suma_op termino | termino
suma_op -> + | - | ++ | --
termino -> termino mult_op factor | factor
mult_op -> * | / | %
factor -> factor pot_op componente | componente
pot_op -> ^
componente -> ( expresion ) | numero | id | bool | op_logico componente
op_logico -> && | || | !
cadena -> "cualquier texto"
```

Correspondencia Métodos del Parser Gramática

M todo	Producci n gramatical
parse() / program()	programa -> main { lista_declaracion }
variable_declaration()	declaracion_variable -> tipo identificador ;
statement()	sentencia -> seleccion iteracion repeticion ...
assignment()	asignacion -> id = sent_expresion
if_statement()	seleccion -> if expresion then ... end
while_statement()	iteracion -> while expresion lista_sentencias end
do_while_statement()	repeticion -> do lista_sentencias while expresion
input_statement()	sent_in -> cin >> id ;
output_statement()	sent_out -> cout << salida
expression()	expresion -> expresion_simple [rel_op expresion_simple]
simple_expression()	expresion_simple -> expresion_simple suma_op termino termino
term()	termino -> termino mult_op factor factor
factor()	factor -> factor pot_op componente componente
component()	componente -> (expresion) numero id bool op_logico

3. Requisitos Técnicos Cumplidos

#	Requisito	Estado	Descripción
1	Entrada desde archivo de tokens	Cumplido	load_tokens_from_file() lee tokens.txt con tipo, lexema, línea y columna
2	Análisis sintáctico y construcción del AST	Cumplido	Parser descendente recursivo, genera AST con nodos tipados
3	Visualización gráfica del árbol	Cumplido	GraphicalASTView: nodos circulares conectados por líneas (QGraphicsView), zoom y pan
4	Visualización colapsable del árbol	Cumplido	ASTTableView: tabla HTML con ramas unicode (), colores por tipo
5	Reporte de errores con línea y columna	Cumplido	"Error sintáctico: mensaje (línea X, columna Y)"
6	Continuación tras errores no fatales	Cumplido	synchronize() avanza hasta SEMICOLON / END / RBRACE y retoma el análisis
7	Archivo errores_sintacticos.txt	Cumplido	save_errors() genera el archivo al finalizar cada análisis
8	2 archivos válidos + 1 con errores	Cumplido	test_valido1.txt, test_valido2.txt, test_errores.txt

4. Archivos de Prueba

test_valido1.txt Declaraciones, if/else, cout

```
main {  
  int x, y;  
  float z;  
  bool bandera;  
  
  x = 10;  
  y = x + 5;  
  z = y * 2.5;  
  bandera = true;  
  
  if x < y then  
    cout << "x es menor que y" << x;  
  else  
    cout << "x no es menor";  
  end  
  
  cout << "z vale" << z;  
}
```

57 tokens reconocidos | 0 errores lógicos | 0 errores sintácticos

test_valido2.txt while, do-while, cin, if sin else

```
main {  
  int contador;  
  bool activo;  
  
  contador = 0;  
  activo = true;  
  
  while contador < 10  
  cout << "Iteracion" << contador;  
  contador = contador + 1;  
  end  
  
  do  
  cin >> contador;  
  cout << "Leiste" << contador;  
  while contador != 0  
  
  if contador == 0 then  
  cout << "Contador es cero";  
  end  
}
```

AST generado correctamente | 0 errores sint cticos

test_errores.txt Errores sintácticos intencionales

```
main {  
  int x y; /* Error 1: falta coma/; entre x e y */  
  float ; /* Error 2: falta identificador */  
  x = 10 /* Error 3: falta ; */  
  if x < then /* Error 4: falta operando derecho */  
  cout << "error sin operando derecho";  
  end  
  while x > 0  
  x = x - 1;  
} /* Error 5+: falta end en while */
```

Errores detectados (7):

Se esperaba 'SEMICOLON' encontrado 'y' (línea 2, columna 9)
Se esperaba 'ID' encontrado ';' (línea 3, columna 9)
Se esperaba 'SEMICOLON' encontrado 'x' (línea 4, columna 3)
Factor inválido encontrado 'then' (línea 5, columna 10)
Se esperaba 'THEN' encontrado 'cout' (línea 6, columna 5)
Se esperaba 'END' encontrado '}' (línea 10, columna 1)
Se esperaba 'end' para cerrar el while (línea 10, columna 1)

5. árbol Sintáctico Abstracto (AST)

AST test_valido1.txt

```
PROGRAMA [1:1]  
MAIN: main [1:1]  
DECLARACIONES [1:1]  
DECL_VAR [2:3] -> TIPO:int | ID:x | ID:y  
DECL_VAR [3:3] -> TIPO:float | ID:z  
DECL_VAR [4:3] -> TIPO:bool | ID:bandera  
SENTENCIAS [1:1]  
ASIGNACION: x [6:3] -> ID:x | NUMERO:10  
ASIGNACION: y [7:3] -> ID:y | OP_SUMA:+ (ID:x, NUMERO:5)  
ASIGNACION: z [8:3] -> ID:z | OP_MULT:* (ID:y, REAL:2.5)  
ASIGNACION: bandera [9:3]-> ID:bandera | BOOL:true  
IF [11:3]  
CONDICION -> OP_REL:< (ID:x, ID:y)  
ENTONCES -> SALIDA (CADENA:"x es menor que y", ID:x)  
SINO -> SALIDA (CADENA:"x no es menor")  
SALIDA [17:3] -> CADENA:"z vale" | ID:z
```

AST test_valido2.txt

```
PROGRAMA [1:1]  
MAIN: main [1:1]  
DECLARACIONES [1:1]  
DECL_VAR [2:3] -> TIPO:int | ID:contador  
DECL_VAR [3:3] -> TIPO:bool | ID:activo  
SENTENCIAS [1:1]  
ASIGNACION: contador [5:3] -> NUMERO:0  
ASIGNACION: activo [6:3] -> BOOL:true  
MIENTRAS [8:3]  
CONDICION -> OP_REL:< (ID:contador, NUMERO:10)
```

```
CUERPO -> SALIDA | ASIGNACION:contador+1
HACER_MIENTRAS [13:3]
CUERPO -> ENTRADA(ID:contador) | SALIDA
CONDICION -> OP_REL:!= (ID:contador, NUMERO:0)
IF [18:3]
CONDICION -> OP_REL:== (ID:contador, NUMERO:0)
ENTONCES -> SALIDA (CADENA:"Contador es cero")
```

6. Manejo de Errores

El parser implementa recuperaci3n de errores mediante el m3todo `synchronize()`:

- Registra el error con tipo, mensaje, l3nea y columna exactos
- Avanza tokens hasta encontrar un punto de sincronizaci3n: SEMICOLON, END O RBRACE
- Retoma el an3lisis desde ese punto permite detectar m3ltiples errores en una sola pasada
- Al finalizar, genera `errores_sintacticos.txt` con todos los errores encontrados

Los errores se visualizan en la terminal Err. Sint. del IDE: tarjetas con borde naranja sobre fondo negro, mostrando descripci3n del error y posici3n l3nea:columna.

7. Descripci3n del IDE

El analizador sint3ctico est3 integrado en un IDE completo desarrollado en Python 3 + PyQt6:

Característica	Descripci3n
Editor de c3digo	Resaltado de sintaxis, n3meros de l3nea, auto-indentaci3n, Ctrl+D / Ctrl+I
Vista gr3fica del AST	Nodos circulares conectados por l3neas, zoom Ctrl+rueda, pan con mouse
Vista colapsable del AST	Tabla HTML con ramas unicode, colores por tipo de nodo, fondo oscuro
Terminal de Tokens	Tabla HTML: tipo coloreado por categor3a, lexema, l3nea y columna
Terminal Err. L3xicos	Tarjetas con borde rojo: s3mbolo no reconocido y posici3n exacta
Terminal Err. Sint.	Tarjetas con borde naranja: descripci3n del error y posici3n l3nea:col
Consola principal	Salida con badges de estado (info, ok, warning, error)
15 temas visuales	dark_pro, pure_black, tokyo_night, catppuccin, dracula, nord, monokai
Paleta de comandos	Ctrl+Shift+P b3squeda y ejecuci3n de cualquier acci3n del IDE
Drag & Drop	Arrastrar archivos .txt / .c / .cpp directamente al editor

Tecnolog3as utilizadas

- Python 3.11
- PyQt6 6.x interfaz gr3fica multiplataforma
- Analizador l3xico propio `compiler/lexer.py`
- Analizador sint3ctico propio `descendente recursivo compiler/parser.py`
- Sin dependencias externas para el an3lisis (solo stdlib + PyQt6)